

Research

Transition

1 What is a Transition in Front End?

A **transition** is a visual effect that smoothly changes a UI element from one state to another over time.

Example:

- Color change on hover
- Button size increase
- Card sliding into view
- Fade in / fade out

2 CSS Transitions (Most Common)

◆ Definition

CSS transitions allow property changes to happen **smoothly over a duration**.

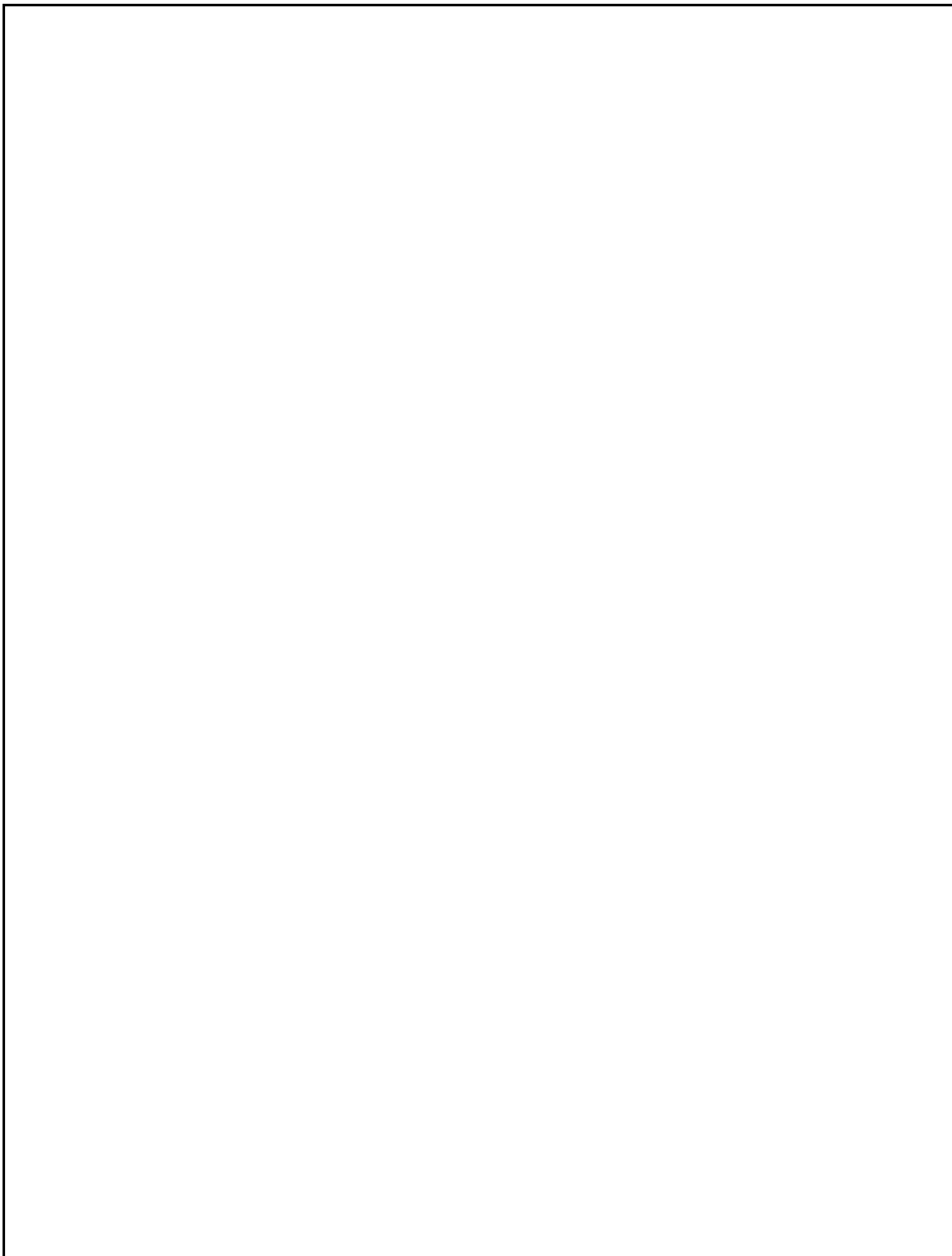
◆ Basic Syntax

transition: property duration timing-function delay;

◆ Example

```
button {  
  background-color: blue;  
  transition: background-color 0.3s ease;  
}
```

```
button:hover {  
  background-color: red;  
}
```



◆ Common Transition Properties

- color
- background-color
- width / height
- opacity
- transform
- box-shadow

3 Transition Properties Explained

Property	Description
transition-property	What to animate
transition-duration	How long it takes
transition-timing-function	Speed curve
transition-delay	Delay before start

Timing Functions

- linear
- ease
- ease-in
- ease-out
- ease-in-out
- cubic-bezier()

4 Transform + Transition (Very Important 🚀)

transform is **performance-friendly** and commonly used.

```
.card {  
  transition: transform 0.3s ease;  
}
```

```
.card:hover {  
  transform: scale(1.1);  
}
```

Common transforms:

- scale()
- translateX(), translateY()
- rotate()
- skew()

5 CSS Animations vs Transitions

Transitions	Animations
Triggered by state change	Run automatically
Simple effects	Complex sequences
Easier	More control

6 JavaScript & Transitions

JS can:

- Add/remove classes
- Control transition timing
- React to user events

```
element.classList.add("active");
```

Used when:

- Click events
- Scroll animations
- Dynamic UI updates

7 Front-End Frameworks & Transitions

◆ React

- CSS transitions
- react-transition-group
- Framer Motion

◆ Vue

- <transition> component
- Built-in animation hooks

◆ Angular

- Angular Animations module

8 Best Practices

- ✓ Use transform & opacity
- ✓ Keep transitions short (0.2s–0.4s)
- ✓ Avoid animating width and height when possible
- ✓ Ensure accessibility (reduce motion)

Keywords You Can Search

Use these for deeper learning:

- **CSS transitions**
 - **transition property CSS**
 - **CSS transform animation**
 - **frontend animations**
 - **hover effects CSS**
 - **UI micro-interactions**
 - **performance friendly CSS animations**
-

Transform

1 What is transform in Front End?

transform is a CSS property used to **change the position, size, or rotation of an element without affecting the normal layout.**

- ✓ Very smooth
- ✓ High performance
- ✓ Used with transitions & animations

2 translate – Move Elements

◆ What it does

Moves an element **from its original position.**

◆ Syntax

transform: translate(x, y);

◆ Examples

/ Move right 50px, down 20px */*

transform: translate(50px, 20px);

/ Move only X */*

transform: translateX(100px);

/ Move only Y */*

transform: translateY(-30px);

◆ Common Use Cases

- Centering elements
- Slide-in animations
- Hover effects

- Modals & sidebars

3 rotate – Rotate Elements

◆ What it does

Rotates an element **clockwise or counterclockwise**.

◆ Units

- deg (degrees)
- rad
- turn

◆ Syntax

```
transform: rotate(45deg);
```

◆ Example

```
.icon {  
  transition: transform 0.3s ease;  
}
```

```
.icon:hover {  
  transform: rotate(180deg);  
}
```

◆ Real Uses

- Menu icons
- Arrows
- Loading indicators

scale – Resize Elements

◆ What it does

Increases or decreases size **without breaking layout**.

◆ Syntax

```
transform: scale(x, y);
```

◆ Examples

```
/* Increase size */
```

```
transform: scale(1.2);
```

```
/* Shrink */
```

```
transform: scale(0.8);
```

```
/* Scale only X or Y */
```

```
transform: scaleX(1.5);
```

```
transform: scaleY(0.7);
```

◆ Real Uses

- Button hover effects
- Card focus
- Image zoom

Combining Transforms

You can use **multiple transforms together**:

```
.card:hover {  
  transform: translateY(-10px) scale(1.05) rotate(2deg);  
}
```

6 transform-origin

Controls **where the transform starts from**.

```
transform-origin: center;  
transform-origin: top left;
```

Example:

```
.box {  
  transform-origin: left center;  
  transform: rotate(45deg);  
}
```

7 2D vs 3D Transforms (Quick Look)

```
transform: translateZ(50px);  
transform: rotateY(45deg);
```

Used for:

- Cards flip
- Parallax effects
- 3D UI components

8 Performance Best Practices 🚀

- ✓ Prefer transform over top/left
- ✓ Combine with transition or animation
- ✓ Avoid overuse on mobile
- ✓ Support reduced motion

```
@media (prefers-reduced-motion: reduce) {  
  * {  
    transform: none;  
    transition: none;  
  }  
}
```

```
}
```

9 Quick Comparison Table

Transform	Purpose
translate	Move
rotate	Rotate
scale	Resize
skew	Tilt
matrix	Advanced

10 Example Mini Project

```
.card {  
  transition: transform 0.3s ease;  
}  
  
.card:hover {  
  transform: translateY(-10px) scale(1.05);  
}
```

@Keyframes

1 What is @keyframes?

@keyframes is used to **define CSS animations** by specifying **how an element changes over time**.

Unlike transition:

- Animations can run **automatically**
- Can have **multiple steps**
- Can **loop, reverse, or pause**

2 Basic Syntax

```
@keyframes animation-name {  
  from {  
    /* starting state */  
  }  
  to {  
    /* ending state */  
  }  
}
```

Or using percentages:

```
@keyframes moveBox {  
  0% {  
    transform: translateX(0);  
  }  
  50% {  
    transform: translateX(100px);  
  }  
  100% {  
    transform: translateX(0);  
  }  
}
```

3 Using Animation with Elements

```
.box {  
  animation-name: moveBox;  
  animation-duration: 2s;  
}
```

Shorthand

```
animation: moveBox 2s ease-in-out infinite;
```

4 Animation Properties Explained

Property	Description
animation-name	Name of keyframes
animation-duration	Time to complete
animation-timing-function	Speed curve
animation-delay	Delay before start
animation-iteration-count	Number of repeats
animation-direction	Normal / reverse
animation-fill-mode	Keeps end state
animation-play-state	Running / paused

5 Common Timing Functions

- linear
- ease
- ease-in
- ease-out
- ease-in-out
- cubic-bezier()

6 Real Examples

◆ Fade In

```
@keyframes fadeIn {  
  from { opacity: 0; }  
  to { opacity: 1; }  
}  
  
.element {  
  animation: fadeIn 1s ease;  
}
```

◆ Loading Spinner

```
@keyframes spin {  
  from { transform: rotate(0); }  
  to { transform: rotate(360deg); }  
}  
  
.loader {  
  width: 40px;  
  height: 40px;  
  border: 4px solid gray;  
  border-top: 4px solid blue;  
  border-radius: 50%;  
  animation: spin 1s linear infinite;  
}
```

◆ Bounce Effect

```
@keyframes bounce {  
  0%, 100% { transform: translateY(0); }  
  50% { transform: translateY(-30px); }  
}  
  
.ball {  
  animation: bounce 0.8s ease infinite;
```

```
}
```

7 @keyframes with Transform 🚀

Most animations use transform for performance:

```
@keyframes slideUp {  
  from {  
    transform: translateY(50px);  
    opacity: 0;  
  }  
  to {  
    transform: translateY(0);  
    opacity: 1;  
  }  
}
```

8 Best Practices ✅

- ✓ Use transform & opacity
- ✓ Keep animations short
- ✓ Avoid infinite animations for important UI
- ✓ Respect accessibility

```
@media (prefers-reduced-motion: reduce) {  
  * {  
    animation: none !important;  
  }  
}
```

9 Transition vs Animation

Transition	Animation
Needs trigger	Runs automatically
Simple	Complex

One step	Multi-step
----------	------------

Animation

1 What is Animation in Front End?

Animation in front-end development means adding **movement and visual change over time** to elements to:

- Improve user experience
- Give feedback (hover, click, loading)
- Make UI feel alive

2 CSS Animation = @keyframes + animation

An animation has **two main parts**:

1. @keyframes → defines *what happens*
2. animation → controls *how it runs*

3 Basic Animation Example

```
@keyframes fadeIn {  
  from {  
    opacity: 0;  
    transform: translateY(30px);}  
  to {  
  
    opacity: 1;  
    transform: translateY(0);  
  }  
}
```

```
.box {  
  animation: fadeIn 1s ease-out;  
}
```

Animation Properties (Important)

Property	Purpose
animation-name	Name of keyframes
animation-duration	How long
animation-timing-function	Speed curve
animation-delay	Wait before starting
animation-iteration-count	Repeat times
animation-direction	Reverse / alternate
animation-fill-mode	Keep styles
animation-play-state	Pause / play

Shorthand

animation: name duration timing delay iteration direction fill-mode;

5 Common Animation Timing Functions

- linear
- ease
- ease-in
- ease-out
- ease-in-out
- cubic-bezier(x1, y1, x2, y2)
- steps(n)

6 Most Used Animations

◆ Loading Spinner

```
@keyframes spin {  
  to { transform: rotate(360deg); }  
}  
.loader {  
  animation: spin 1s linear infinite;  
}
```

◆ Pulse Effect

```
@keyframes pulse {  
  0% { transform: scale(1); }
```

```
50% { transform: scale(1.1); }  
100% { transform: scale(1); }  
}  
.btn {  
  animation: pulse 1.5s infinite;  
}
```

◆ Slide In

```
@keyframes slideIn {  
  from { transform: translateX(-100px); }  
  to { transform: translateX(0); }  
}  
.card {  
  animation: slideIn 0.6s ease-out;  
}
```

7 Animation vs Transition

Animation	Transition
Auto-run	Needs trigger
Multi-step	One-step
Loops	No loops
Complex	Simple

8 Performance Best Practices 🚀

- ✓ Use transform & opacity
- ✓ Avoid top, left, width, height
- ✓ Keep durations short ($\leq 1s$)
- ✓ Avoid infinite animations on important UI

9 Accessibility (Very Important 🗣️)

Support reduced motion:

```
@media (prefers-reduced-motion: reduce) {  
  * {  
    animation: none !important;  
  }  
}
```

Attribute Selectors

1 What are Attribute Selectors?

Attribute selectors select HTML elements **based on their attributes or attribute values**, not their tag or class.

They are very useful when:

- You can't add classes
- Working with dynamic HTML
- Styling form inputs
- Targeting specific links or data attributes

2 Basic Attribute Selector

- ◆ Select elements that have an attribute

```
input[type] {  
  border: 2px solid blue;  
}
```

→ Selects **all inputs** that have a type attribute.

3 Exact Attribute Value Selector (=)

```
input[type="text"] {  
  background-color: lightyellow;  
}
```

✓ Matches exact value only.

4 Attribute Contains Value (~=)

Matches if the value contains a **word**.

```
[class~="btn"] {  
  padding: 10px;  
}
```

✓ class="btn primary" → matched

✗ class="btn-primary" → not matched

5 Attribute Starts With (^=)

```
a[href^="https"] {  
  color: green;  
}
```

✓ Selects secure links

6 Attribute Ends With (\$=)

```
a[href$=".pdf"] {  
  color: red;  
}
```

✓ Selects PDF download links

7 Attribute Contains (*=)

Matches anywhere in value.

```
img[src*="logo"] {  
  border: 2px solid black;  
}
```

8 Case-Insensitive Attribute Selector

```
input[type="email"] i {  
  border-color: blue;  
}
```


🔗 Data Attributes (Very Important ★)

HTML:

```
<div data-role="admin"></div>  
<div data-role="user"></div>
```

CSS:

```
div[data-role="admin"] {  
  background-color: gold;  
}
```

Used heavily in:

- JavaScript frameworks
- Component-based UI
- Dynamic content

10 Attribute Selector Table

Selector	Meaning
[attr]	Has attribute
[attr="value"]	Exact match
[attr~="val"]	Contains word
[attr^="val"]	Starts with
[attr\$="val"]	Ends with
[attr*="val"]	Contains

1 1 Attribute Selectors + Pseudo-Classes

```
input[type="checkbox"]:checked {  
  accent-color: green;  
}
```

1 2 Real-World Examples

✓ Style required fields

```
input[required] {  
  border-left: 4px solid red;  
}
```

✓ Disable styles

```
button[disabled] {  
  opacity: 0.5;  
  cursor: not-allowed;  
}
```

1 3 Best Practices ✓

- ✓ Use when classes are unavailable
 - ✓ Prefer classes for large projects
 - ✓ Avoid overcomplex selectors
 - ✓ Combine with data-* attributes
-

Pseudo-elements

1 What are Pseudo-Elements?

Pseudo-elements allow you to **style or create parts of an element** that don't exist directly in HTML.

👉 `::before` and `::after` create **virtual elements** inside an element.

2 `::before` and `::after` Basics

◆ Syntax

```
.element::before {  
  content: "";  
}
```

```
.element::after {  
  content: "";  
}
```

⚠ **content is required**, even if empty.

3 Where Do They Appear?

- `::before` → **first child**
- `::after` → **last child**

Inside the selected element.

4 Common Uses 🎯

- ✓ Decorative icons
- ✓ Lines & separators
- ✓ Overlays
- ✓ Badges & labels

- ✓ Tooltips
- ✓ Background effects

5 Simple Example

```
h2::before {  
  content: " ★ ";  
}
```

Result:

★ Heading

6 Creating Shapes with ::before

```
.box {  
  position: relative;  
}
```

```
.box::after {  
  content: "";  
  position: absolute;  
  width: 100%;  
  height: 4px;  
  background: red;  
  bottom: 0;  
  left: 0;  
}
```

→ Creates a bottom line under .box

7 ::before + ::after with Hover

```
.btn {  
  position: relative;  
  overflow: hidden;  
}  
  
.btn::after {  
  content: "";  
  position: absolute;  
  width: 100%;  
  height: 100%;  
  background: black;  
  left: -100%;  
  top: 0;  
  transition: transform 0.3s ease;  
  transform: translateX(0);  
}  
  
.btn:hover::after {  
  transform: translateX(100%);  
}
```

8 Using with content: attr()

```
<button data-text="Click Me"></button>
```

```
button::after {  
  content: attr(data-text);  
}
```

9 Z-Index & Positioning 🚀

```
.card {  
  position: relative;  
}
```

```
.card::before {
```

```
content: "";
position: absolute;
inset: 0;
background: rgba(0,0,0,0.1);
z-index: -1;
}
```

10 ::before vs :before

- ✓ Modern CSS uses ::before
- ✓ Single colon works for backward compatibility

1 1 Limitations ✗

- ✗ Can't be added to void elements (img, input)
 - ✗ Only two per element
 - ✗ Not readable by screen readers (mostly decorative)
-

Pseudo-classes

1 What are Pseudo-classes?

Pseudo-classes select elements based on their **state or position**, not their tag or class.

They respond to:

- User interaction 
- Keyboard navigation 
- Element order in the DOM

Syntax:

```
selector:pseudo-class {
  /* styles */
}
```

```
}
```

2 :hover — When Mouse Is Over

◆ What it does

Applies styles **when the user moves the mouse over an element**.

◆ Example

```
button:hover {  
  background-color: darkblue;  
  color: white;  
}
```

◆ Common Uses

- Button effects
- Links highlight
- Card lift effects

 Works on **desktop**, limited on mobile

3 :focus — When Element Is Selected

◆ What it does

Applies styles when an element is:

- Clicked
- Selected using **Tab**

```
input:focus {  
  border-color: blue;
```



```
outline: none;
}
```

◆ Important for Accessibility

Keyboard users rely on focus indicators.

✅ Better practice:

```
input:focus-visible {
  outline: 2px solid blue;
}
```

:active — When Clicked

◆ What it does

Applies **while the element is being clicked**.

```
button:active {
  transform: scale(0.95);
}
```

Used for:

- Button press feedback
- Mobile touch effects

5 :nth-child() — Select by Position

◆ Syntax

```
:nth-child(n)
```

◆ Examples

```
li:nth-child(1) {  
  color: red;  
}
```

```
li:nth-child(odd) {  
  background: #eee;  
}
```

```
li:nth-child(even) {  
  background: #ccc;  
}
```

```
li:nth-child(3n) {  
  color: green;  
}
```

6 nth-child() Formula Explained

`:nth-child(an + b)`

Expression	Meaning
$2n$	Every 2nd element
$2n+1$	Odd
$3n$	Every 3rd
$3n+1$	Start from 1

7 :nth-child() vs :nth-of-type()

```
p:nth-child(2)    /* second child overall */  
p:nth-of-type(2) /* second <p> only */
```

⚠ This is a common mistake!

8 Combining Pseudo-classes 🔗

```
a:hover:focus {  
  color: red;  
}
```

```
li:nth-child(odd):hover {  
  background-color: yellow;  
}
```

9 Order Matters (IMPORTANT 🚨)

For links use **LVHA** order:

```
a:link  
a:visited  
a:hover  
a:active
```

10 Real-World UI Example

```
.card {  
  transition: transform 0.3s ease;  
}
```

```
.card:hover {
```

```
transform: translateY(-10px);  
}
```

```
.card:active {  
  transform: scale(0.98);  
}
```

Best Practices

- ✓ Always add focus styles
- ✓ Avoid removing outline without replacement
- ✓ Use hover + focus together
- ✓ Test with keyboard
- ✓ Use nth-child carefully

Quick Summary Table

Pseudo-class	Use
:hover	Mouse over
:focus	Keyboard/click
:active	While clicking
:nth-child()	Position

